1) Sorting Algorithms

	Best	Average	Worst	Space	Notes.
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Stable
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n^2)$	Unstable, minimal swaps
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Stable, fast on nearly sorted
Merge Sort	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$	$O(n)$	Stable, D & C
Quick Sort	$O(n\log n)$	$O(n\log n)$	$O(n^2)$	$O(\log n)$	Unstable.
Heap Sort	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$	$O(1)$	Unstable, in-place
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	Stable, k = range of values
Radix Sort	$O(d(n+k))$	—	—	$O(n+k)$	d = digits, k = base

Data Structures

	Insert	Delete	Search	Access	Notes.
Array	$O(n)$	$O(n)$	$O(n)$	$O(1)$	Cache-friendly
Sorted Array	$O(n)$	$O(n)$	$O(\log n)$	$O(1)$	Binary search, expensive inserts
Linked lists	$O(1)$	$O(1)$	$O(n)$	$O(n)$	Sequential access
Binary Heap	$O(\log n)$	$O(\log n)$	$O(n)$	$O(1)$*	*min-max only. Build = $O(n)$
BST (Balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	Ordered operation AVL & Black Trees
Hash Table	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	N/A	$O(n)$ worst case, no ordering
Union-find	$O(\alpha(n))$	N/A	$O(\alpha(n))$	N/A	Path compression + Union by rank.
Stack	$O(1)$	$O(1)$	$O(n)$	$O(1)$*	*Top only, LIFO DFS
Queue	$O(1)$	$O(1)$	$O(n)$	$O(1)$*	*Front only, FIFO BFS

Recurrences

Recurrence	Solution	Example.
$T(n) = T(n/2) + O(1)$	$O(\log n)$	Binary Search
$T(n) = 2T(n/2) + O(n)$	$O(n \log n)$	Merge Sort
$T(n) = 2T(n/2) + O(1)$	$O(n)$	Tree traversal
$T(n) = T(n/2) + O(n)$	$O(n)$	Median finding
$T(n) = 3T(n/2) + O(n)$	$O(n^{1.585})$	Karatsuba
$T(n) = 4T(n/2) + O(n)$	$O(n^2)$	Naive multiply
$T(n) = 2T(n/2) + O(n^2)$	$O(n^2)$	Root dominates
$T(n) = 7T(n/2) + O(n^2)$	$O(n^{2.807})$	Strassen's
$T(n) = 9T(n/3) + O(n)$	$O(n^2)$	Leaves Dominate

Graph Algorithms

→ Undirected graph with n nodes:
- Maximum edges $= n(n-1)/2$
- A tree has exactly $n-1$ edges (connected graph)
- A forest with k components has exactly $n-k$ edges.

→ Directed graph with n nodes:
- Maximum edges $= n(n-1)$ (each pair has 2 directions)
- Maximum edges with self loop $= n^2$

Handshaking Lemma → In any finite undirected graph, the sum of all vertex degrees is exactly twice the no. of edges

$$\sum_{v \in V} \deg(v) = 2|E|.$$

V = vertices, E = edges, $O(V+E)$ → Adjacency list space (Sparse)

Adjacency matrix: $O(V^2)$ space (dense)

Algorithm	Time	Space	Structure	Key Properties.
BFS	$O(V+E)$	$O(V)$	Queue	Shortest path (unweighted) Level-order
DFS	$O(V+E)$	$O(V)$	Stack/ Rec	Cycle detection, Topological Sort, SCCs
Topological Sort	$O(V+E)$	$O(V)$	Stack	DAGs Only, Reverse DFS post-order
Dijkstra's	$O(V+E(\log V))$	$O(V)$	Min-heap	No-negative weights Detects neg. cycles.
Bellman-Ford	$O(V \cdot E)$	$O(V)$	Array	Handles negative cycles.
Floyd-Warshall	$O(V^2)$	$O(V^2)$	2D-Array	APSP, DP on intermediate vertices
Prim's MST	$O(V+E(\log V))$	$O(V)$	Min-heap	Greedy, Grow tree from vertex
Kruskal's MST	$O(E \log E)$	$O(V)$	Union-Find	Greedy, Sorts edges, Cycle Property.

GREEDY ALGORITHMS

Make locally optimal choices. Must prove correctness via "greedy stays ahead". Does NOT always work

Algorithm	Time	Space	Key Insights.
→ Interval Scheduling	$O(n \log n)$	$O(n)$	Sort by finish time. Pick the earliest one first. (Greedy stays ahead)
→ Dijkstra's Algorithm	$O((V+E)\log V)$	$O(V)$	Extract closely unvisited vertex. Relax neighbors (Does not work with neg. cycles)
→ Prim's MST	$O(V+E(\log V))$	$O(V)$	Grow tree, always add cheapest edge. Cut property guarantees correctness.
→ Kruskal's MST	$O(E \log E)$	$O(V)$	Sort all edges, Add cheapest edge that doesn't make a cycle (union-find)
→ Huffman Coding	$O(n \log n)$	$O(n)$	Repeatedly merge two lowest-frequency nodes. Produces optimal prefix-free code.
→ Activity Selection	$O(n \log n)$	$O(n)$	Sort by finish time, greedy Select.
→ Fractional knapsack	$O(n \log n)$	$O(n)$	Sort by value/weight ratio. Take greedy. (Does NOT work for 0/1 knapsack)

Dynamic Programming

- Remember solutions to overlapping subproblems. Define subproblems, write recurrences (Bellman eq^n). Identify base cases, determine fill order. extract the answer.

Problem	Time	Space	Subproblem & Recurrence.
→ Fibonacci	$O(n)$	$O(1)$	$F[i] = F[i-1] + F[i-2]$. Base: $F[0]=0$, $F[1]=1$.
→ Weighted Interval Scheduling	$O(n \log n)$	$O(n)$	$OPT[j] = \max(W_j + OPT[P(j)], OPT[j-1])$ Sort by finish. $P(j)$ = last compatible.
→ 0/1 Knapsack	$O(nW)$	$O(nW)$	$OPT[i][w] = \max(OPT[i-1][w], v_i + OPT[i-1][w_i - w_i])$. Pseudopolynomial. NP-Hard.
→ Longest Common Subsequence	$O(mn)$	$O(mn)$	if $x_i = y_j$: $LCS[i-1][j-1] + 1$. Else : $\max(LCS[i-1][j], LCS[i][j-1])$.
→ Edit Distance	$O(mn)$	$O(mn)$	$OPT[i][j] = \min(OPT[i-1][j]+1, OPT[i][j-1]+1, OPT[i-1][j-1]+\delta)$
→ Bellman Ford (SSSP)	$O(V \cdot E)$	$O(V)$	$dist[v][i] = \min$ over (u,v) of $\{dist[u][i-1] + w(u,v)\}$ i = max # edges. Neg. cycle improves at v
→ Floyd-Warshall (APSP)	$O(V^3)$	$O(V^2)$	$dist[i][j][k] = \min(dist[i][j][k-1], dist[i][k][k-1] + dist[k][j][k-1])$ All pair shortest path
→ Matrix-chain Multiplication	$O(n^3)$	$O(n^2)$	$OPT[i][j] = \min$ over k of $\{OPT[i][k] + OPT[k+1][j] + P_i P_k P_j\}$. Optimal parenthisization.

What Techniques to use?

→ Overlapping subproblems + Overlapping Substructure → Dynamic Programming

→ Problem splits into independent halves - Divide & Conquer

→ Irrevocable local choices produce global optimum - Greedy Algorithm.

→ Shortest path in unweighted graph - BFS

→ Shortest path, weighted, no negative edges - Dijkstra's (Greedy + Heap)

→ Shortest path with negative edges - Bellman-Ford (DP)

→ All-pair shortest path - Floyd-warshall (DP)

→ MST - Prim's / Kruskal's (Greedy)

→ Need repeated min/max extraction - Binary Heap / Priority Queues.

→ Dynamic Connectivity / Grouping - Union-Find

→ Counting / ordering / scheduling - Sorting + Greedy / DP